

TerraFly: Complete Technical & Educational Master Guide

Zero-to-Expert Architecture, Data Pipelines, 3D Elevation Systems, and Codebase Textbook

EXECUTIVE SUMMARY & MISSION STATEMENT

TerraFly is an interactive, browser-based, high-integrity geospatial and computer vision workbench. It is designed to transform optical overhead imagery (satellite photos, aerial surveys, drone captures) and digital elevation models (DEMs) into interactive, textured, 3D surface models with accurate height analysis, point-to-point elevation inspection, slope calculation, and structural visualization.

Built for the **Smart India Hackathon (SIH 2026 / PS 26175)**, TerraFly solves a foundational engineering and scientific challenge: **How can analysts, defense personnel, and geospatial researchers visualize and inspect 3D terrain and urban structures from overhead imagery while remaining mathematically honest about what is an absolute measurement versus what is an AI-estimated visual proxy?**

SECTION 1 — WHAT IS TERRAFLY?

1.1 The Core Problem TerraFly Solves

When viewing a flat 2D satellite photograph (such as a Google Earth image, Sentinel-2 capture, or drone orthophoto), human eyes can intuitively perceive mountains, valleys, and buildings due to shading, shadows, and perspective. However, standard computer images (PNG, JPEG) are simply 2D arrays of colour pixels (Red, Green, Blue). **They contain zero vertical depth information (Z-axis).**

In traditional workflows, acquiring 3D terrain requires expensive LiDAR flights, complex multi-view photogrammetry pipelines (taking dozens of overlapping photos from different angles), or specialized satellite radar missions. When only a single satellite image or a coarse digital elevation model is available, analysts face severe hurdles:

- 1. The Single-Photo Dilemma:** A single 2D image cannot physically determine absolute elevation in metres without independent ground truth.
- 2. The "Vibe-Coding" Illusion:** Many modern AI tools take an image, predict relative depth, slap an arbitrary "meters" label on the vertical axis, and claim scientific accuracy. This is dangerous in military, disaster-response, and engineering operations.
- 3. The Texture/Geometry Mismatch:** Standard elevation viewers either stretch aerial textures vertically down steep cliffs (creating ugly "texture drips"), or smooth away sharp mountain ridges and building boundaries.

1.2 Who Would Use TerraFly?

- **Disaster Response & Relief Teams:** Assessing landslide slopes, flood water accumulation paths, and road accessibility across mountain terrain.
- **Urban Planners & Civil Engineers:** Rapidly visualizing building heights, terrain grades, and structural footprints before commissioning ground surveys.
- **Defense & Tactical Planners:** Inspecting terrain elevation, sightlines, high ground, and approach slopes from reconnaissance imagery.
- **Geospatial & Remote Sensing Students:** Learning how coordinate reference systems (CRS), GeoTIFF rasters, monocular depth estimation, affine calibration, and WebGL rendering interact.

1.3 What the User Gives to the Application (Inputs)

TerraFly offers **two distinct operational workflows**:

1. Workflow A: DEM Terrain (Measured Mountain Mode - Recommended)

- **Input 1:** A georeferenced optical GeoTIFF (**.tif** / **.tiff**) containing RGB colour imagery.
- **Input 2:** A georeferenced single-band Digital Elevation Model (DEM) GeoTIFF (**.tif** / **.tiff**) containing real elevation values (in metres).
- **Input 3:** Metadata declaring the DEM Source (e.g., **NASA SRTMGL1 v3**) and Vertical Datum (e.g., **EGM96 orthometric**).

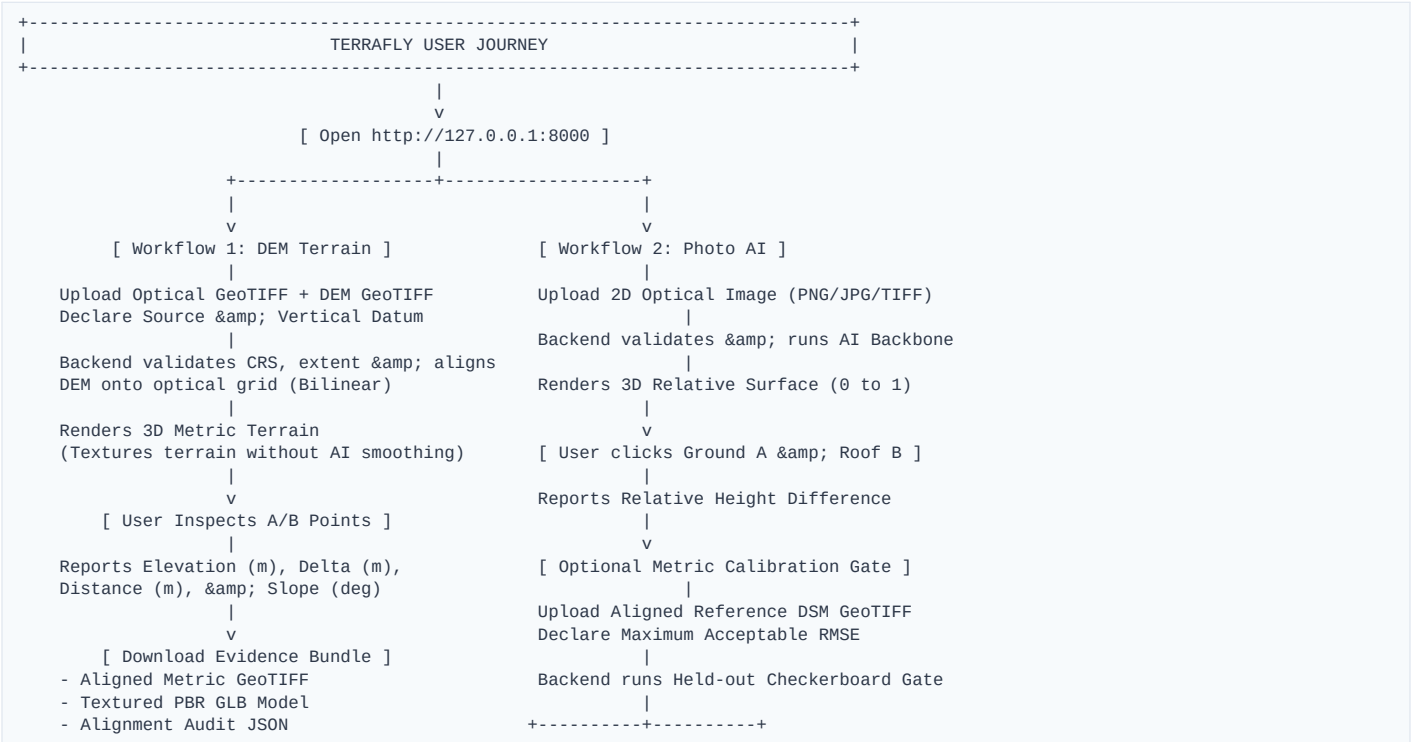
2. Workflow B: Photo AI (Monocular Relative Depth Mode - Experimental)

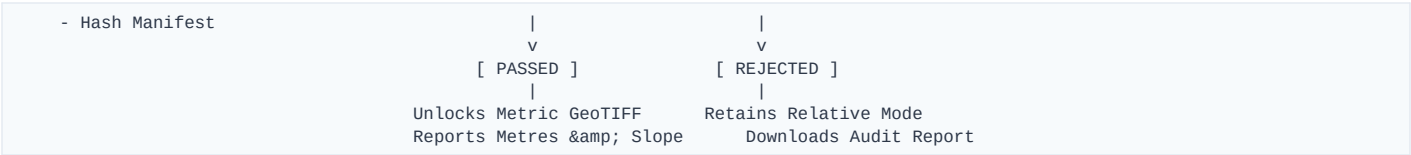
- **Input 1:** Any standard 2D image (**.png**, **.jpg**, **.jpeg**, or uncalibrated **.tif**).
- **Optional Calibration Input:** An independently surveyed, pixel-aligned Reference DSM GeoTIFF or surveyed Ground Control Points (GCPs) with a declared Root Mean Square Error (RMSE) tolerance.

1.4 What the Application Produces (Outputs)

- **Interactive 3D Viewport:** An orbitable, flyable 3D WebGL mesh rendered with Three.js, supporting real-time lighting adjustments, wireframe toggles, elevation colour thermal ramps, and optional 3D building extrusions.
- **A/B Point Inspection & Analysis:** Real-time sampling of ground elevation (Point A) vs. roof/ridge elevation (Point B), reporting:
 - Relative height difference (\$0.0\$ to \$1.0\$) OR calibrated metric elevation difference in metres (\$m\$).
 - Horizontal ground distance in metres (\$m\$) (for projected CRS inputs).
 - True terrain slope angle in degrees (\$\theta^\circ\$).
- **Standardized GLB 2.0 3D Mesh:** Exportable **.glb** file with embedded PNG textures, smooth unit normals, PBR lighting, and neutral synthetic steep-wall rendering.
- **Scientific Evidence Bundle:** Downloadable, SHA-256 hashed data files including:
 - **relative_surface.npy** (lossless NumPy float32 array).
 - **metric_surface.tif** (lossless georeferenced 32-bit floating point GeoTIFF in metres).
 - **metric_analysis_grid.json** (precise coordinate grid matching 3D vertices).
 - **calibration_report.json** or **terrain_alignment_report.json** (complete mathematical audit trail).
 - **job_manifest.json** (cryptographic record of all inputs, model revisions, and outputs).

1.5 The Complete User Journey





1.6 TerraFly Actual Architecture Flowchart



SECTION 2 — COMPLETE PROJECT STRUCTURE

Let us explore every single directory and tracked source file in the TerraFly project.

```

G:\TerraFly_FINAL_WINDOWS\
■■■■ .env.example           # Sample environment variable definitions
■■■■ .gitattributes         # Git line-ending normalization config
■■■■ .gitignore            # Git ignore list (hides .venv, weights, runtime jobs)
■■■■ BUILD_LOG.md          # Chronological milestone development log
■■■■ Check-TerraFly.cmd    # One-click Windows batch verification utility
■■■■ DATA_SOURCES.md      # Dataset provenance, hashes, and licensing records
■■■■ DECISIONS.md          # Comprehensive Architectural Decision Records (ADRs)
■■■■ FILE_GUIDE.md         # Complete repository ledger describing every tracked file
■■■■ FOUR_DAY_PLAN.md      # Risk-mitigation project planning schedule
■■■■ LEARNING_NOTES.md     # Educational notes on depth, height, and GeoTIFFs
■■■■ LIMITATIONS.md        # Scientific boundaries and disclaimers
■■■■ PROJECT_STATUS.md     # Exact status of implemented vs pending components
■■■■ pyproject.toml        # Python package configuration and dependencies
■■■■ README.md             # Primary project overview and quickstart guide
■■■■ requirements-ml-cu130.txt # Pinned PyTorch CUDA 13.0 dependencies
■■■■ RESULTS.md            # Quantitative benchmark results and verification logs
■■■■ Start-TerraFly.cmd    # One-click Windows application launcher
■■■■ THIRD_PARTY_NOTICES.md # Licenses for third-party models and libraries
■■■■
■■■■ backend/              # BACKEND PYTHON PACKAGE
■   ■■■■ terrafly/        # Core application module
■   ■   ■■■■ __init__.py  # Package initialization and version definition
■   ■   ■■■■ artifacts.py # 3D GLB export, display grids, diagnostics, SHA-256
■   ■   ■■■■ calibration.py # Reference DSM & GCP robust calibration math & gates
■   ■   ■■■■ config.py    # Pydantic/dataclass application settings & limits
■   ■   ■■■■ evaluation.py # Masked RMSE, MAE, bias, and Pearson evaluation
■   ■   ■■■■ imaging.py   # Safe image decoding, GeoTIFF CRS/transform parsing
■   ■   ■■■■ jobs.py      # Job lifecycle, JSON persistence, and disk storage
■   ■   ■■■■ main.py      # FastAPI route controllers and static frontend mount
■   ■   ■■■■ pipeline.py  # Orchestrator for Photo AI analysis workflow
■   ■   ■■■■ schemas.py   # Pydantic data schemas and API contracts
■■■■
■■   ■■■■ terrain.py      # DEM Terrain alignment, reprojection, and normalization
■■   ■
■■   ■■■■ inference/      # Machine learning model adapters
■■   ■   ■■■■ __init__.py # Package initializer
■■   ■   ■■■■ base.py     # InferenceAdapter protocol and Prediction dataclass
■■   ■   ■■■■ conventions.py # Inverse-depth to relative-height conversion logic
■■   ■   ■■■■ depth_anything_v2.py # Real Depth Anything V2 HuggingFace adapter
■■   ■   ■■■■ deterministic.py # Fast deterministic test adapter (non-AI proxy)
■■   ■   ■■■■ factory.py  # Adapter factory with test-adapter safety interlock
■■   ■   ■■■■ tiling.py   # Bounded overlapping tile slicer and blender
■■   ■
■■   ■■■■ tests/          # BACKEND AUTOMATED TEST SUITE
■■   ■   ■■■■ __init__.py # Test package initializer
■■   ■   ■■■■ conftest.py # Pytest fixtures, test client, and synthetic images
■■   ■   ■■■■ test_adapter_contract.py # Tests for height convention, orientation, and safety
■■   ■   ■■■■ test_api.py # API endpoint validation, upload limits, error handling
■■   ■   ■■■■ test_artifacts.py # GLB structure, PBR materials, diagnostics, structure test
■■   ■   ■■■■ test_calibration.py # Reference DSM & GCP calibration math and gate rejection
■■   ■   ■■■■ test_evaluation.py # Masked mathematical metrics unit tests
■■   ■   ■■■■ test_geotiff.py # GeoTIFF CRS/affine preservation tests
■■   ■   ■■■■ test_terrain.py # DEM terrain alignment, overlap, and NoData tests
■■   ■   ■■■■ test_tiling.py # Overlapping tile geometry and blending math tests
■■   ■
■■■■ frontend/           # FRONTEND REACT / TYPESCRIPT / THREE.JS APP
■   ■■■■ index.html      # HTML5 application shell mount point
■   ■■■■ package.json     # Node.js project manifest and script definitions
■   ■■■■ package-lock.json # Exact npm dependency lockfile
■   ■■■■ tsconfig.json    # TypeScript project root configuration
■   ■■■■ tsconfig.app.json # Browser TypeScript compiler options
■   ■■■■ tsconfig.node.json # Vite Node.js configuration TypeScript options
■   ■■■■ vite.config.ts   # Vite bundler, development server, and test setup
■   ■

```

```

■ ■■■ dist/ # Production build output (served by FastAPI)
■ ■ ■■■ index.html # Compiled production HTML
■ ■ ■■■ assets/ # Minified JavaScript bundle and CSS styles
■ ■
■ ■■■ src/ # Frontend application source code
■ ■■■ api.ts # Typed API client for FastAPI backend communications
■ ■■■ App.tsx # Main React Workbench component and workflow manager
■ ■■■ App.test.tsx # React UI component tests
■ ■■■ main.tsx # React 19 application DOM bootstrap
■ ■■■ styles.css # Handcrafted design system and responsive CSS styles
■ ■■■ surfaceGeometry.ts # Pure Three.js geometry, bilinear sampling, and slope math
■ ■■■ SurfaceViewer.tsx # Interactive Three.js WebGL canvas and controller
■ ■■■ SurfaceViewer.test.ts # Unit tests for 3D geometry and sampling logic
■ ■■■ test-setup.ts # Vitest DOM environment setup
■ ■■■ types.ts # TypeScript type definitions mirroring backend schemas
■
■ ■■■ sample_data/ # BUNDLED TEST FIXTURES & DEMO DATA
■ ■■■ LICENSE.txt # CC0 public domain dedication for sample fixtures
■ ■■■ README.md # Explanation of bundled synthetic fixtures
■ ■■■ terraflly_calibration_demo_input.tif # Georeferenced RGB test image
■ ■■■ terraflly_calibration_demo_metadata.json # Metadata for calibration fixture
■ ■■■ terraflly_calibration_demo_reference.tif # Aligned synthetic reference DSM
■ ■■■ terraflly_synthetic_aerial.png # 2D synthetic aerial test image
■ ■■■ terraflly_terrain_demo_dem.tif # Synthetic 20m mountain DEM GeoTIFF
■ ■■■ terraflly_terrain_demo_imagery.tif # Synthetic 10m optical GeoTIFF
■ ■■■ terraflly_terrain_demo_metadata.json # Metadata for mountain terrain fixture
■
■ ■■■ scripts/ # AUTOMATION, LAUNCHERS, AND VERIFICATION
■ ■■■ benchmark_height_model.py # Standalone model benchmark script
■ ■■■ compare_model_variants.py # Comparison script for Small vs Large models
■ ■■■ create_calibration_demo.py # Generator for synthetic calibration fixtures
■ ■■■ create_handoff_manifest.py # Cryptographic manifest generation script
■
■ ■■■ create_offline_sample.py # Deterministic offline sample generator
■ ■■■ create_terrain_demo.py # Generator for synthetic mountain DEM fixture
■ ■■■ package_release.ps1 # Production release packaging script
■ ■■■ setup.ps1 # Full machine setup (venv, npm build, checks)
■ ■■■ smoke_final_workflow.py # End-to-end Python test for calibration path
■ ■■■ smoke_real_api.py # API smoke test on live server
■ ■■■ smoke_real_model.py # Direct PyTorch inference smoke test
■ ■■■ smoke_terrain_workflow.py # End-to-end Python test for DEM Terrain path
■ ■■■ smoke_tiled_model.py # Multi-tile tiled inference test
■ ■■■ start.ps1 # Production single-server launcher script
■ ■■■ verify.ps1 # Comprehensive project verification script
■
■ ■■■ docs/ # DOCUMENTATION & SPECIFICATIONS
■ ■■■ ARCHITECTURE.md # High-level architecture and evidence map
■ ■■■ DEMO_SCRIPT.md # 6-8 minute presentation sequence for judges
■ ■■■ JUDGE_QA.md # Comprehensive defense Q&A for hackathon judges
■ ■■■ MODEL_METRIC_CARD.md # Truthful model evaluation card
■ ■■■ MODEL_UPGRADE_EVALUATION.md # Audit of depth models and upgrade justifications
■ ■■■ OPERATOR_GUIDE.md # Step-by-step user manual and flight controls
■ ■■■ PS_REQUIREMENTS_TRACEABILITY.md # Traceability matrix for SIH PS 26175
■ ■■■ REAL_TERRAIN_DATA_GUIDE.md # Guide to sourcing real Sentinel-2 & SRTM DEM data
■ ■■■ TEAM_TECHNICAL_GUIDE.md # Deep technical guide for team members
■ ■■■ TRAINING_ROADMAP.md # Future ML training roadmap (DFC23 dataset)
■ ■■■ UX_RATIONALE.md # Design rationale and accessibility considerations
■ ■■■ cookbook/ # Cookbook recipes
■ ■■■ COOKBOOK_SOURCE_INDEX.md # Source index for cookbook references
■ ■■■ TERRAFLY_COOKBOOK.md # In-depth technical cookbook
■
■ ■■■ runtime/ # MACHINE RUNTIME STORAGE (Ignored by Git)
■ ■■■ jobs/ # Per-job directories storing uploads & generated artifacts

```

SECTION 3 — TECHNOLOGY STACK

Technology	Category	What It Is in Simple Words	Why TerraFly Uses It	Where It Appears in Code	Connection to Other Tech
Python 3.12	Programming Language	A popular, readable programming language with powerful math and AI libraries.	Powers backend orchestration, geospatial processing, and machine learning inference.	Entire backend/ and scripts/ directory	Drives FastAPI, PyTorch, Rasterio, NumPy

Technology	Category	What It Is in Simple Words	Why TerraFly Uses It	Where It Appears in Code	Connection to Other Tech
FastAPI	Web Framework	A modern, high-performance web framework for building APIs in Python.	Provides fast, typed HTTP REST endpoints with automatic JSON validation and background task execution.	<code>backend/terrafly/main.py</code>	Receives requests from Frontend, dispatches to Pipeline/Jobs
Pydantic v2	Data Validation	A library that enforces strict data types and schema validation using Python type hints.	Ensures inputs (coordinates, thresholds, filenames) and JSON outputs adhere to exact data contracts.	<code>backend/terrafly/schemas.py</code>	Validates FastAPI route inputs and job manifest serialization
PyTorch (CUDA 13)	Deep Learning Framework	A tensor computation and neural network engine with GPU acceleration.	Executes the Depth Anything V2 foundation model on NVIDIA RTX graphics cards (with CPU fallback).	<code>backend/terrafly/inference/depth_anything_v2.py</code>	Loaded by Transformers, feeds predictions into NumPy
Hugging Face Transformers	ML Model Library	A repository and API for downloading and running state-of-the-art pretrained AI models.	Loads the Depth-Anything-V2-Large-hf vision transformer checkpoint and image preprocessor.	<code>backend/terrafly/inference/depth_anything_v2.py</code>	Interacts with Hugging Face Hub, outputs PyTorch tensors
Rasterio / GDAL	Geospatial Raster Engine	A Python library for reading, writing, and projecting georeferenced satellite images and DEMs.	Extracts Coordinate Reference Systems (CRS), affine transforms, pixel resolutions, and reprojects DEMs.	<code>backend/terrafly/imaging.py</code> , <code>terrain.py</code> , <code>calibration.py</code>	Bridges raw GeoTIFF bytes to NumPy arrays
NumPy	Numerical Computing	The standard Python library for fast array and matrix mathematical operations.	Performs percentile clipping, affine regression fitting, bilateral filtering, and array manipulation.	All files in <code>backend/terrafly/</code>	Core data representation between rasterio, PyTorch, and JSON
Pillow (PIL)	Image Processing	Python Imaging Library for opening, converting, resizing, and saving 2D raster images.	Handles PNG/JPEG decoding, colorization palette mapping, and 16-bit grayscale height texture encoding.	<code>backend/terrafly/imaging.py</code> , <code>artifacts.py</code>	Converts between NumPy arrays and PNG images
React 19	Frontend Framework	A JavaScript/TypeScript library for building declarative user interfaces from components.	Manages the interactive state of the workbench, file pickers, progress bars, and analysis panels.	<code>frontend/src/App.tsx</code> , <code>main.tsx</code>	Renders UI DOM, coordinates with Three.js canvas
TypeScript	Programming Language	JavaScript with static typing, catching bugs before code runs.	Guarantees data type consistency between FastAPI backend schemas and frontend UI components.	Entire <code>frontend/src/</code> directory	Compiles to JavaScript via Vite
Three.js	3D Graphics Library	A JavaScript library that renders 3D graphics in the browser using WebGL/WebGPU.	Generates the 3D surface mesh, applies textures, computes lighting/shadows, and handles 3D raycasting.	<code>frontend/src/SurfaceViewer.tsx</code> , <code>surfaceGeometry.ts</code>	Renders WebGL canvas inside the React DOM
Vite	Frontend Build Tool	A fast frontend development server and bundler.	Bundles TypeScript and React into highly optimized, minified production assets in <code>frontend/dist/</code> .	<code>frontend/vite.config.ts</code>	Compiles frontend for FastAPI static hosting

SECTION 4 — FRONTEND ARCHITECTURE & COMPONENTS

4.1 How the Frontend Starts and Mounts

1. **HTML Shell (frontend/index.html)**: The browser requests `http://127.0.0.1:8000/`. FastAPI intercepts this and serves `frontend/dist/index.html`. The HTML contains a single `<div id="root"></div>` mount point and loads the compiled JavaScript asset bundle.

2. **React Bootstrap (frontend/src/main.tsx)**:

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

3. **Application Workbench (frontend/src/App.tsx)**: Mounts the primary application state, initializes topbar branding, workflow selectors, drag-and-drop file inputs, progress steppers, and result views.

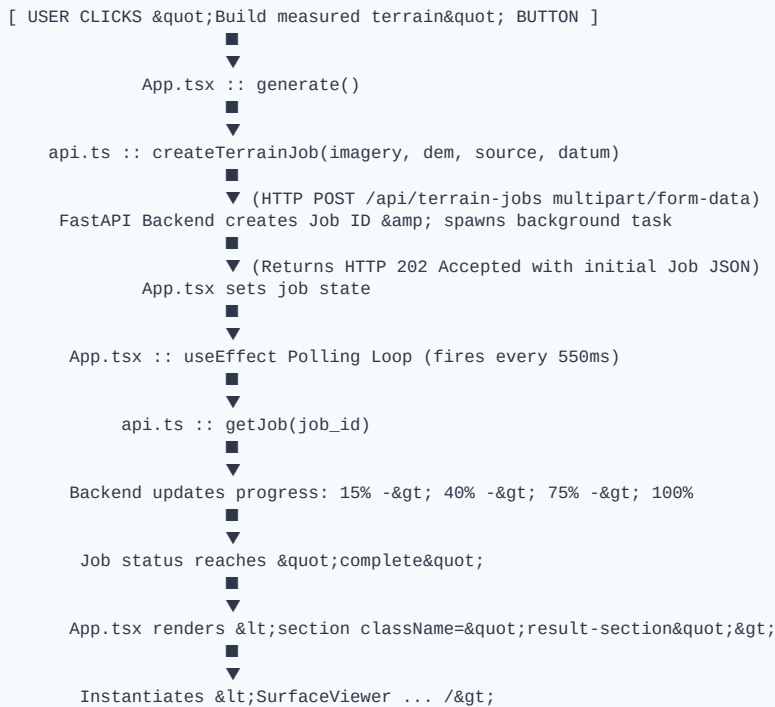
4.2 State Management in App.tsx

The main component maintains several synchronized React state variables:

- **workflow**: Current active tab ("**terrain**" for DEM Terrain or "**photo**" for Photo AI).
- **file**: Selected optical image (**File** | **null**).
- **demFile**: Selected DEM elevation file (**File** | **null**, used in DEM Terrain).
- **terrainSource** & **terrainDatum**: User-entered provenance strings for the DEM.
- **job**: Active or completed job object (**Job** | **null**, containing manifest, stage, progress, artifacts).
- **busy**: Boolean indicating whether an upload or calculation is currently in progress.
- **points**: Array of inspected 3D surface points (**InspectedPoint[]**, max 2 points: Point A and Point B).
- **referenceFile**, **sourceDescription**, **verticalDatum**, **maxRmse**: State for the metric calibration form.

4.3 Detailed Component Architecture: User Action to UI Update

Let us trace what happens when a user clicks "**Build measured terrain**":



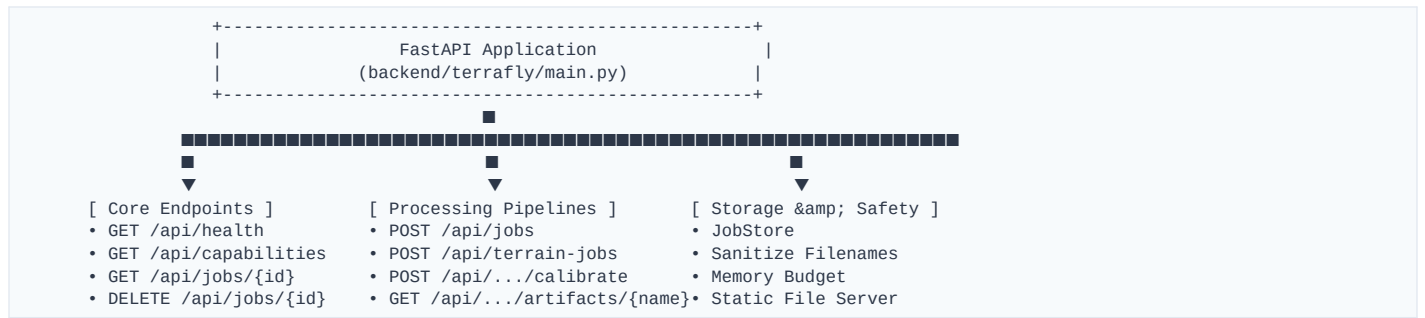
4.4 The 3D Surface Viewer (**SurfaceViewer.tsx**)

The **SurfaceViewer** is the interactive 3D core of TerraFly. It encapsulates a full Three.js scene:

- **WebGLRenderer**: Initialized with antialiasing and sRGB colour space.
- **PerspectiveCamera**: 42-degree field of view positioned at coordinate $(X=8, Y=7, Z=9)$ looking at $(0, 0.8, 0)$.
- **Lighting Rig**:
- **HemisphereLight**: Ambient ground-sky bounce lighting (intensity 2.3).
- **DirectionalLight** ("Sun"): Positioned on a circular orbit determined by the user's **Sun direction slider** (default azimuth 35°).
- **Dual Material Grouping**:
- **surfaceMaterial** (**THREE.MeshStandardMaterial**): Receives the optical photograph texture OR vertex elevation colours.
- **wallMaterial** (**THREE.MeshStandardMaterial**): A synthetic neutral dark-slate material (sRGB: $0.34, 0.39, 0.37$) assigned to steep triangles where height difference $\Delta z \geq 0.055$. This prevents top-down satellite pixels from being stretched vertically along cliffs.
- **Controls**:
- **OrbitControls**: Left-click rotates, right-click pans, mouse-wheel zooms.
- **PointerLockControls**: Activated in **First-person mode**. The mouse looks around, while **W/A/S/D** moves forward/back/left/right, **Q/E** moves down/up, and **Shift** provides a speed boost.

SECTION 5 — BACKEND ARCHITECTURE & REST APIS

The backend is built with FastAPI and organized into modular domain components.



5.1 Complete Endpoint Specification

Endpoint 1: GET `/api/health`

- **Purpose:** Verifies that the TerraFly backend is running.
- **Input:** None.
- **Output:** JSON `{"status": "ok", "service": "TerraFly", "version": "1.0.0"}`.
- **File & Function:** `backend/terrafly/main.py` -> `health()`.

Endpoint 2: GET `/api/capabilities`

- **Purpose:** Informs the frontend about upload limits, supported file formats, tile sizes, and calibration requirements.
- **Input:** None.
- **Output:** **Capabilities** schema JSON (Max upload bytes: 25MB, Max pixels: 50MP, Tile size: 1024, Overlap: 128).
- **File & Function:** `backend/terrafly/main.py` -> `capabilities()`.

Endpoint 3: POST `/api/jobs`

- **Purpose:** Creates an asynchronous Photo AI inference job from an uploaded 2D image.
- **Input:** Multipart form-data with file field **upload**.
- **Processing Flow:**
 1. `read_upload()` reads stream and enforces the 25MB file limit.
 2. `inspect_image()` checks format (PNG/JPEG/TIFF), dimensions, decompression bombs, and geospatial CRS.
 3. `estimated_working_bytes()` computes working memory ($\text{width} \times \text{height} \times 32$ bytes). If $\text{width} \times \text{height} > 768 \text{ MB}$, returns HTTP 413.
 4. `JobStore.create()` assigns a 32-character hexadecimal UUID, writes initial `job.json`, and saves raw input image.
 5. Enqueues `run_job` into FastAPI **BackgroundTasks**.
- **Output:** HTTP 202 Accepted with **JobManifest** JSON.
- **File & Function:** `backend/terrafly/main.py` -> `create_job()`.

Endpoint 4: POST `/api/terrain-jobs`

- **Purpose:** Creates a DEM Terrain job combining an optical GeoTIFF with a single-band DEM GeoTIFF.
- **Input:** Multipart form-data with **imagery**, **dem**, **source_description**, **vertical_datum**, **vertical_units**.
- **Processing Flow:**
 1. Validates both GeoTIFFs via `inspect_image()` and `inspect_source_dem()`.
 2. Verifies both files have valid geospatial Coordinate Reference Systems (CRS).
 3. Enforces that vertical units are metres.

4. Enqueues **run_source_dem_job** into background tasks.

- **Output:** HTTP 202 Accepted with **JobManifest** JSON (Scientific State: **Metric Source DEM**).
- **File & Function:** **backend/terrafly/main.py** -> **create_terrain_job()**.

Endpoint 5: **GET /api/jobs/{job_id}**

- **Purpose:** Polls the execution status, stage, progress percentage, warnings, and artifact list of an active job.
- **Input:** Path parameter **job_id** (32-character hex).
- **Output:** **JobManifest** JSON.
- **File & Function:** **backend/terrafly/main.py** -> **get_job()**.

Endpoint 6: **POST /api/jobs/{job_id}/calibrate/reference**

- **Purpose:** Evaluates a completed georeferenced Photo AI job against an independently sourced, pixel-aligned Reference DSM GeoTIFF.
- **Input:** Multipart form-data with **reference** file, **source_description**, **vertical_datum**, **max_rmse_m**.
- **Processing Flow:** Calls **calibrate_reference()** in **backend/terrafly/calibration.py**.
- **Output:** Updated **JobManifest** with **Metric Calibrated** (if passed) or **Georeferenced Relative** (if rejected).
- **File & Function:** **backend/terrafly/main.py** -> **calibrate_with_reference()**.

Endpoint 7: **GET /api/jobs/{job_id}/artifacts/{artifact_name}**

- **Purpose:** Securely downloads a generated artifact file (GLB model, GeoTIFF, NPY array, PNG preview, JSON grid).
- **Input:** Path parameters **job_id** and **artifact_name**.
- **Security Check:** Path resolution ensures requested file resides strictly within the sanitized **runtime/jobs/{job_id}** directory, preventing directory traversal attacks.
- **Output:** Binary **FileResponse** with appropriate MIME media type.
- **File & Function:** **backend/terrafly/main.py** -> **get_artifact()**.

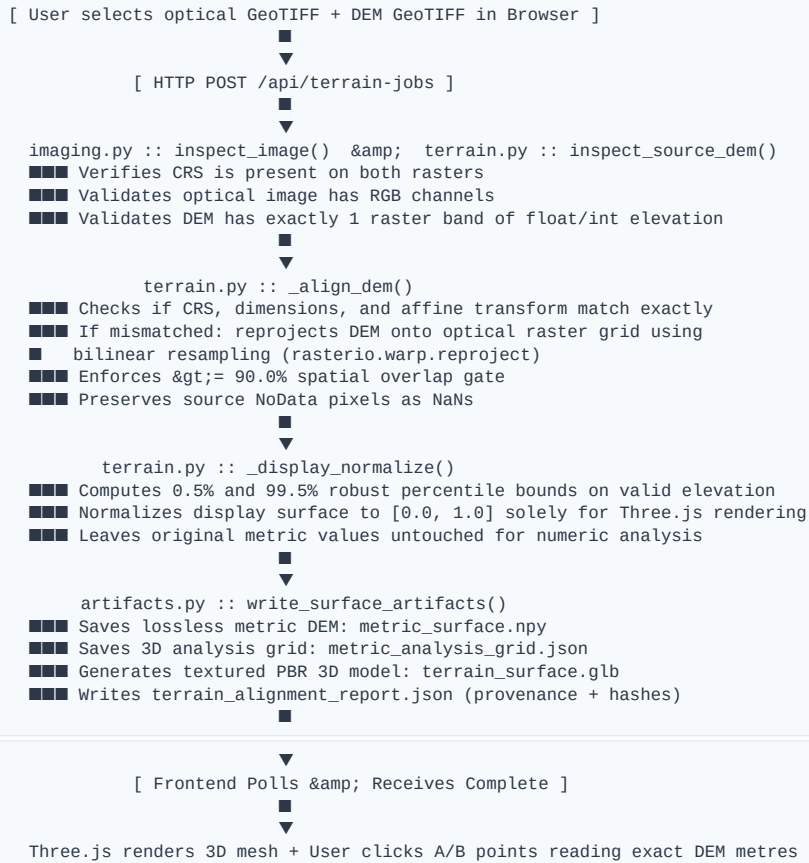
Endpoint 8: **DELETE /api/jobs/{job_id}**

- **Purpose:** Cleans up disk storage by permanently deleting a completed job's directory.
- **Input:** Path parameter **job_id**.
- **Safety Rule:** Rejects deletion with HTTP 409 if the job is currently **queued** or **running**.
- **Output:** HTTP 204 No Content.
- **File & Function:** **backend/terrafly/main.py** -> **delete_job()**.

SECTION 6 — TERRAFLY'S COMPLETE DATA PIPELINE

Let us trace the exact movement of data across memory, disk, algorithms, and network buffers for both workflows.

6.1 The Real DEM Terrain Data Pipeline



6.2 The Real Photo AI Data Pipeline



```

■■■ Slope limiter bounds extreme adjacent vertex deltas
    ■
    ▼
    artifacts.py :: write_surface_artifacts()
■■■ Saves canonical numeric array: relative_surface.npy
■■■ Saves canonical analysis grid: relative_grid.json
■■■ Saves display-only mesh grid: display_grid.json
■■■ Exports textured 3D model: relative_surface.glb
■■■ Generates height_diagnostics.json (recording tilt & cleanup)

```

SECTION 7 — IMAGE GENERATION & COMPUTER VISION PROCESSING

7.1 Image Input Sources and Formats

TerraFly supports four primary image input categories:

- Standard 2D Aerial Photographs (.png, .jpg, .jpeg):** Standard RGB photos from drones, aircraft, or mapping portals. Handled via Pillow (**PIL.Image**).
- Georeferenced Optical GeoTIFFs (.tif, .tiff):** Multi-band satellite rasters containing embedded cartographic projections (e.g., UTM Zone 43N) and affine bounding boxes. Handled via **rasterio**.
- Single-band Thermal / Grayscale Imagery (.tif, .png):** 1-channel rasters. TerraFly automatically repeats the single band across 3 channels (\$RGB = [L, L, L]\$) while issuing a visible domain warning: *Thermal/TIR imagery is outside the model's optical training domain*.
- Single-band Digital Elevation Models (.tif, .tiff):** 1-channel 32-bit floating-point or 16-bit integer elevation rasters.

7.2 Byte Decoding and Security Sanitization

In **backend/terrafly/imaging.py**, raw upload bytes undergo a 4-tier security filter before reaching any computer vision model:

- Filename Sanitization (sanitize_filename):** Rejects null bytes (`\x00`), forward slashes (`/`), and backslashes (`\\`). Strips directory components and enforces length ≤ 180 characters.
- Decompression Bomb Guard:** Wraps Pillow image loading in **warnings.catch_warnings()** and elevates **Image.DecompressionBombWarning** to a hard exception, preventing malicious images from exhausting server RAM.
- Size & Pixel Budget:** Rejects uploads > 25 MB and rasters $> 50,000,000$ pixels.
- Working Memory Budget (estimated_working_bytes):** Computes $\text{width} \times \text{height} \times 32$ bytes. If processing would exceed 768 MB of RAM, the job is rejected before inference begins.

7.3 Pixel Transformation & Guided Display Filtering

In **backend/terrafly/artifacts.py**, TerraFly employs four sophisticated computer vision filtering algorithms exclusively for the 3D display mesh, leaving the underlying numeric array completely untouched:

1. Isolated Outlier Spike Removal (_remove_isolated_outliers)

A 3×3 sliding window calculates the local median and Median Absolute Deviation (MAD):

$$\text{MAD} = \text{median}(|x_i - \text{median}(X)|)$$

$$\text{Threshold} = \max(0.075, 6.0 \times \text{MAD} + 0.015)$$

Any pixel deviating from its local median by more than the threshold with ≤ 3 supporting neighbours is replaced by the local median.

2. Joint RGB-Guided Bilateral Filter (_joint_bilateral_filter)

Smooths noisy depth estimates across flat areas while preserving sharp object boundaries by weighting neighbours according to spatial distance, height difference, and RGB colour similarity:

$$W(p, q) = \exp\left(-\frac{\|p - q\|^2}{2\sigma_s^2} - \frac{\|I_p - I_q\|^2}{2\sigma_c^2} - \frac{\|z_p - z_q\|^2}{2\sigma_z^2}\right)$$

where $\sigma_s = 1.35$ (spatial), $\sigma_c = 0.14$ (colour), and $\sigma_z = 0.10$ (height).

3. Connected-Component Rooftop Flattening (`_flatten_rooftop_candidates`)

Fits a least-squares plane to the scene, subtracts it to detect raised rooftop candidates (≥ 80 percentile residual), groups 8-connected raised pixels into components, and sets each rooftop region to its internal median elevation.

4. Adjacent-Slope Limiter (`_limit_display_slopes`)

Iteratively clamps adjacent vertex height deltas to a maximum of 0.5\$ relative units across 4 passes, eliminating rendering artifacts on steep cliffs without affecting measurements.

SECTION 8 — HEIGHT, ELEVATION, & DIFFERENCE CALCULATIONS

8.1 DEM Terrain Mathematical Calculation

In **DEM Terrain mode**, elevation values originate directly from the uploaded DEM raster.

When a user clicks Point A (e.g., ground) and Point B (e.g., mountain peak):

1. Pixel Coordinates Sampling:

The click raycast intersects the 3D surface at UV coordinate (u, v) . This maps to pixel coordinate:

$$\text{column} = u \times (\text{width} - 1), \quad \text{row} = (1 - v) \times (\text{height} - 1)$$

2. Bilinear Elevation Interpolation:

In `frontend/src/surfaceGeometry.ts` -> `sampleMeasurementValue()`, the elevation is interpolated from the four surrounding DEM grid cells:

$$z(x, y) = (1 - \Delta x)(1 - \Delta y) z_{00} + \Delta x (1 - \Delta y) z_{10} + (1 - \Delta x)\Delta y z_{01} + \Delta x \Delta y z_{11}$$

3. Height Difference Calculation:

$$\Delta H = |z_B - z_A| \quad (\text{in metres})$$

4. Horizontal Ground Distance Calculation:

Using the GeoTIFF's 6-element affine transformation matrix:

$$\begin{bmatrix} X_{\text{geo}} \\ Y_{\text{geo}} \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \end{bmatrix} \begin{bmatrix} \text{column} \\ \text{row} \\ 1 \end{bmatrix}$$

$$\Delta X_{\text{geo}} = a(\text{col}_B - \text{col}_A) + b(\text{row}_B - \text{row}_A)$$

$$\Delta Y_{\text{geo}} = d(\text{col}_B - \text{col}_A) + e(\text{row}_B - \text{row}_A)$$

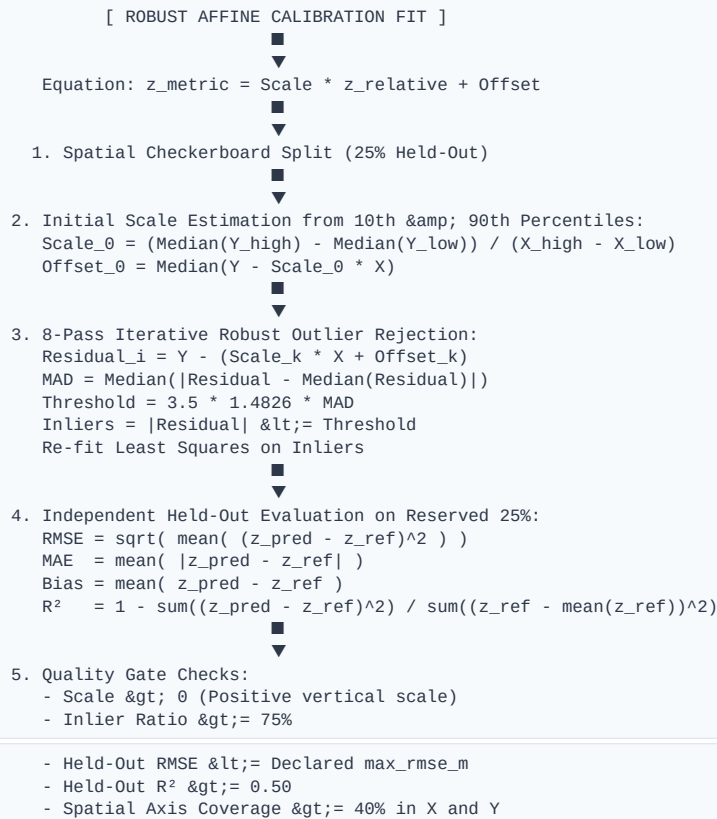
$$D_{\text{horizontal}} = \sqrt{(\Delta X_{\text{geo}})^2 + (\Delta Y_{\text{geo}})^2} \quad (\text{in metres, for projected CRS})$$

5. True Terrain Slope Angle:

$$\text{Slope} = \arctan\left(\frac{\Delta H}{D_{\text{horizontal}}}\right) \times \frac{180^\circ}{\pi}$$

8.2 Photo AI Robust Metric Calibration Mathematics

When calibrating relative Photo AI output ($z_{\text{rel}} \in [0, 1]$) against an aligned reference DSM (z_{ref} in metres):



If all gates pass, **ScientificState** transitions to **Metric Calibrated**, unlocking **metric_surface.tif** and metric A/B inspection. If any gate fails, the run remains **Georeferenced Relative** and exports an honest rejection audit report.

SECTION 9 — MAP & GEOSPATIAL COORDINATE SYSTEM

9.1 Geospatial Concepts for Beginners

- **CRS (Coordinate Reference System)**: A mathematical framework defining how 2D map coordinates relate to real-world locations on the curved Earth.
- **Geographic CRS (e.g., WGS 84 / EPSG:4326)**: Uses spherical angles (**Latitude** in degrees North/South, **Longitude** in degrees East/West). Horizontal distance between degrees varies depending on distance from the equator.
- **Projected CRS (e.g., UTM Zone 43N / EPSG:32643)**: Projects the Earth onto a flat plane using Cartesian units (**metres** Easting and Northing). TerraFly requires projected CRS for horizontal distance and slope calculations.
- **Affine Transformation Matrix**: A 6-parameter formula $[a, b, c, d, e, f]$ mapping pixel columns and rows to real-world map coordinates:
 - a : Pixel width (ground resolution in X direction, e.g. 10.0 m/pixel).
 - b : Rotation/shear term (usually 0.0).
 - c : Upper-left corner X map coordinate.
 - d : Rotation/shear term (usually 0.0).
 - e : Pixel height (negative value in Y direction, e.g. -10.0 m/pixel).
 - f : Upper-left corner Y map coordinate.
- **Vertical Datum**: The zero-elevation reference surface (e.g., Mean Sea Level or the **EGM96** geoid). A height of 500 m is meaningless without knowing whether it is relative to sea level, an ellipsoid, or local ground level.

- **NoData Value:** A special sentinel value (such as **-9999.0**) representing missing data, water bodies, or areas outside the satellite swath.

SECTION 10 — AI, ML, & COMPUTER VISION COMPONENTS

10.1 The Model: Depth Anything V2 Large

TerraFly incorporates the foundation model **Depth Anything V2 Large** (**depth-anything/Depth-Anything-V2-Large-hf**) via Hugging Face Transformers.

- **Model Architecture:** Vision Transformer (ViT-Large) backbone with a Dense Prediction Transformer (DPT) neck and depth head.
- **Input:** Normalized RGB image tensor of shape $(1, 3, H, W)$.
- **Raw Output:** Inverse depth / proximity map tensor of shape $(1, H, W)$.
- **Execution Device:**
 - Automatically selects **cuda** with PyTorch FP16 autocast if an NVIDIA GPU is detected.
 - Automatically catches CUDA Out-Of-Memory (OOM) errors, clears the GPU cache, moves weights to RAM, and retries on **cpu** transparently.

10.2 Tiled Inference Engine (**backend/terrafly/inference/tiling.py**)

To process ultra-high-resolution aerial images without running out of GPU memory or introducing edge seams:

1. **Tile Splitting:** Images $> 4 \times \text{MP}$ are divided into 1024×1024 pixel tiles with a $128 \times \text{pixel}$ overlap.
2. **Affine Overlap Alignment:** Because monocular depth scale and offset can shift between crops, TerraFly performs a least-squares linear fit on overlapping regions:

$$\text{scale}, \text{offset} = \arg\min \sum (\text{tile}_{\text{overlap}} \cdot \text{scale} + \text{offset} - \text{canvas}_{\text{overlap}})^2$$

3. **Cosine Feather Blending:** Overlapping regions are blended using linear/cosine ramp weight masks ($\text{ramp} = [0 \dots 1]$), eliminating visible seam lines.
4. **Global Normalization:** Normalization to $[0.0, 1.0]$ occurs strictly **once** on the assembled full-resolution canvas, preventing contrast stepping between tiles.

10.3 AI Predictions vs. Mathematical Calculations vs. External APIs

Capability	Category	Exact Mechanism in TerraFly	File Location
Relative Surface Estimation	AI Model Inference	Depth Anything V2 Vision Transformer	backend/terrafly/inference/depth_anything_v2.py
DEM Reprojection & Resampling	Deterministic Math	GDAL/Rasterio Bilinear Warp	backend/terrafly/terrain.py
Reference Calibration Fit	Deterministic Math	8-pass Robust Linear Regression ($y = mx + c$)	backend/terrafly/calibration.py
Held-Out RMSE/R ² Validation	Deterministic Math	Mathematical metric computation on masked arrays	backend/terrafly/calibration.py, evaluation.py
3D Normal Vector Generation	Deterministic Math	Central difference gradient ($\nabla z = [\partial z / \partial x, \partial z / \partial y]$)	backend/terrafly/artifacts.py
Optical/DEM Image Acquisition	User Input	User uploads local files directly (no external cloud API)	backend/terrafly/main.py

SECTION 11 — TERRAFLY DICTIONARY: IMPORTANT TERMS

1. API (Application Programming Interface)

- **Simple Meaning:** A structured messenger that allows two different computer programs to talk to each other over a network.
- **How TerraFly Uses It:** The React browser interface calls FastAPI endpoints over HTTP to upload images, check progress, and download 3D models.
- **Where It Appears:** `backend/terrafly/main.py`, `frontend/src/api.ts`.

2. DEM (Digital Elevation Model)

- **Simple Meaning:** A 2D digital grid where every pixel's number represents the physical elevation of the terrain above sea level.
- **How TerraFly Uses It:** Serves as the primary ground-truth geometry source in the DEM Terrain workflow.
- **Where It Appears:** `backend/terrafly/terrain.py`, `sample_data/terrafly_terrain_demo_dem.tif`.

3. DSM (Digital Surface Model)

- **Simple Meaning:** An elevation model that includes the tops of trees, buildings, and structures, rather than just bare ground.
- **How TerraFly Uses It:** Used as the reference ground truth in metric calibration.
- **Where It Appears:** `backend/terrafly/calibration.py`.

4. GeoTIFF

- **Simple Meaning:** A standard TIFF image that contains embedded geographic metadata (real-world GPS coordinates, map projection, pixel size).
- **How TerraFly Uses It:** Preserves spatial positioning and exports calibrated metric height maps.
- **Where It Appears:** `backend/terrafly/imaging.py`, `backend/terrafly/terrain.py`.

5. GLB (Binary glTF 2.0)

- **Simple Meaning:** The standard, portable 3D file format for the web (the 3D equivalent of a JPEG).
- **How TerraFly Uses It:** Exports 3D terrain meshes with embedded textures, normals, and PBR materials.
- **Where It Appears:** `backend/terrafly/artifacts.py` -> `_write_glb()`.

6. PBR (Physically Based Rendering)

- **Simple Meaning:** A computer graphics lighting technique that simulates how light interacts with real-world surfaces.
- **How TerraFly Uses It:** glTF meshes use `pbrMetallicRoughness` materials to ensure natural sunlight and shadow rendering.
- **Where It Appears:** `backend/terrafly/artifacts.py`.

7. RMSE (Root Mean Square Error)

- **Simple Meaning:** The standard mathematical measure of the average error between predicted heights and real heights (in metres).
- **How TerraFly Uses It:** Evaluates whether an AI height map is accurate enough to unlock metric certification.
- **Where It Appears:** `backend/terrafly/calibration.py`, `backend/terrafly/evaluation.py`.

8. Bilinear Interpolation

- **Simple Meaning:** A mathematical method for calculating a smooth value at any sub-pixel point by taking a weighted average of the 4 nearest grid points.
- **How TerraFly Uses It:** Resamples DEMs and samples elevations under user 3D mouse clicks.
- **Where It Appears:** [backend/terrafly/calibration.py](#), [frontend/src/surfaceGeometry.ts](#).

SECTION 12 — FILE-BY-FILE CODE EXPLANATION

backend/terrafly/main.py

- **Purpose:** Main FastAPI web server application, route controllers, and static asset server.
- **Key Functions:**
 - [create_app\(settings\)](#): Initializes FastAPI app, CORS middleware, API routes, and mounts the prebuilt frontend.
 - [create_job\(\)](#): Handles optical photo uploads, validates memory limits, creates job entries, and spawns background AI workers.
 - [create_terrain_job\(\)](#): Handles dual optical+DEM uploads, verifies CRS, and spawns background terrain alignment workers.
 - [calibrate_with_reference\(\)](#): Receives reference DSM GeoTIFF and evaluates calibration gates.
 - [get_artifact\(\)](#): Safely streams generated 3D models and GeoTIFFs to the browser.
- **Calls:** [imaging.py](#), [pipeline.py](#), [terrain.py](#), [calibration.py](#), [jobs.py](#).

backend/terrafly/terrain.py

- **Purpose:** Core engine for the DEM Terrain workflow.
- **Key Functions:**
 - [inspect_source_dem\(data, filename\)](#): Reads raw DEM bytes, verifies single-band structure, and extracts CRS and elevation range.
 - [_align_dem\(imagery_path, dem_path\)](#): Reprojects and resamples DEM onto the optical raster grid, enforcing $\geq 90\%$ overlap.
 - [_display_normalize\(metric\)](#): Computes 0.5% - 99.5% percentile bounds to normalize display mesh without altering metric values.
 - [run_source_dem_job\(job_id, settings\)](#): Orchestrates the entire background DEM alignment, GLB mesh creation, and artifact generation.
- **Calls:** [rasterio.warp.reproject](#), [artifacts.py](#), [jobs.py](#).

backend/terrafly/calibration.py

- **Purpose:** Mathematical validation and quality gates for converting relative surfaces into calibrated metric DSMs.
- **Key Functions:**
 - [robust_affine_fit\(relative, elevation\)](#): Fits $z_{\text{metric}} = \text{Scale} \times z_{\text{rel}} + \text{Offset}$ using 8-pass MAD outlier rejection.
 - [calibrate_reference\(store, manifest, ...\)](#): Performs spatial checkerboard split, fits parameters on training pixels, and validates on held-out pixels.
 - [_gate\(...\)](#): Evaluates scale positivity, inlier ratio ($\geq 75\%$), RMSE threshold, and $R^2 \geq 0.50$.
- **Calls:** [numpy](#), [rasterio](#), [artifacts.py](#).

backend/terrafly/artifacts.py

- **Purpose:** Generates all 2D previews, 3D grids, GLB files, diagnostics, and SHA-256 digests.
- **Key Functions:**
- **build_display_grid(relative_grid, rgb):** Executes outlier replacement, joint bilateral filtering, rooftop flattening, and slope limiting.
- **_write_glb(path, grid, rgb, ...):** Constructs standard glTF 2.0 binary file with embedded PNG texture and split triangle indices.
- **surface_tilt_diagnostics(relative):** Fits a 3D plane to detect perspective tilt without artificially flattening the terrain.
- **reconstruct_structure_candidates(grid):** Finds raised connected components to create optional 3D building extrusions.
- **Calls:** `numpy`, `PIL.Image`, `json`, `struct`.

frontend/src/App.tsx

- **Purpose:** Primary React UI workbench component.
- **Key Functions:**
- **selectWorkflow(mode):** Switches between DEM Terrain and Photo AI tabs.
- **generate():** Dispatches upload requests to backend and initiates progress polling.
- **evaluateCalibration():** Sends reference DSM to calibration endpoint.
- **clearResult():** Deletes job from backend and resets UI state.
- **Calls:** `api.ts`, `SurfaceViewer.tsx`, `surfaceGeometry.ts`.

frontend/src/SurfaceViewer.tsx

- **Purpose:** Three.js WebGL 3D canvas and interaction controller.
- **Key Functions:**
- **chooseNavigation(mode):** Toggles between Orbit camera and First-person flight controls.
- **onPointerUp(event):** Performs 3D raycasting against the terrain mesh to place Point A and Point B markers.
- **animate():** Main render loop driving camera dampening, keyboard flight physics, and WebGL frame rendering.
- **Calls:** `three`, `OrbitControls`, `PointerLockControls`, `surfaceGeometry.ts`.

SECTION 13 — FUNCTION CALL FLOW

```

[ USER CLICKS "Build measured terrain" ]
  ▼
App.tsx :: generate()
  ▼
api.ts :: createTerrainJob()
  ▼ (HTTP POST /api/terrain-jobs)
main.py :: create_terrain_job()
  ▼
  ##### imaging.py :: read_upload()
  ##### imaging.py :: inspect_image()
  ##### terrain.py :: inspect_source_dem()
  ##### jobs.py :: JobStore.create()
  ##### background_tasks.add_task(run_source_dem_job)
    ▼
    terrain.py :: run_source_dem_job()
      ▼
      ##### terrain.py :: _align_dem()
      ##### rasterio.warp.reproject()
      ▼
      ##### terrain.py :: _display_normalize()
      ▼
      ##### artifacts.py :: write_surface_artifacts()
      ##### artifacts.py :: _write_glb()
      ##### artifacts.py :: _colourize()
      ##### artifacts.py :: sha256_file()
      ▼
      ##### calibration.py :: _write_metric_grid()
      ##### terrain.py :: _write_metric_geotiff()
      ▼
      ##### jobs.py :: JobStore.save()

```

SECTION 14 — COMPLETE END-TO-END WALKTHROUGH

Let us walk through a complete, realistic scenario:

1. User Opens Application: The user navigates to <http://127.0.0.1:8000/>. The browser loads the React workbench.

2. User Selects DEM Terrain Workflow: The user clicks the **DEM Terrain** tab (the recommended mountain workflow).

3. File Selection:

- For *Optical texture*, the user selects **terrafly_terrain_demo_imagery.tif** (a 10 m/pixel optical GeoTIFF of a mountain valley in EPSG:32643).
- For *Elevation geometry*, the user selects **terrafly_terrain_demo_dem.tif** (a 20 m/pixel DEM covering the same area).
- User types DEM source: **NASA SRTMGL1 v3** and vertical datum: **EGM96 orthometric**.

4. Initiating Analysis: The user clicks **Build measured terrain**.

5. Backend Processing:

- FastAPI receives both GeoTIFFs, verifies their EPSG:32643 CRS, and reprojects the 20 m DEM onto the 10 m optical grid using bilinear resampling.
- The aligned DEM achieves 100% valid overlap.
- The backend creates **terrain_surface.glb**, **metric_surface.tif**, and **metric_analysis_grid.json**.

6. 3D Visualization:

- The React UI detects job completion and mounts **SurfaceViewer**.
- Three.js renders the 3D mountain landscape textured with real aerial photography.
- Steep mountain ridges retain their natural rock textures because AI smoothing was intentionally disabled.

7. Point Inspection:

- The user clicks a valley floor location (sets **Point A**: Elevation $1,245.30\text{ m}$).

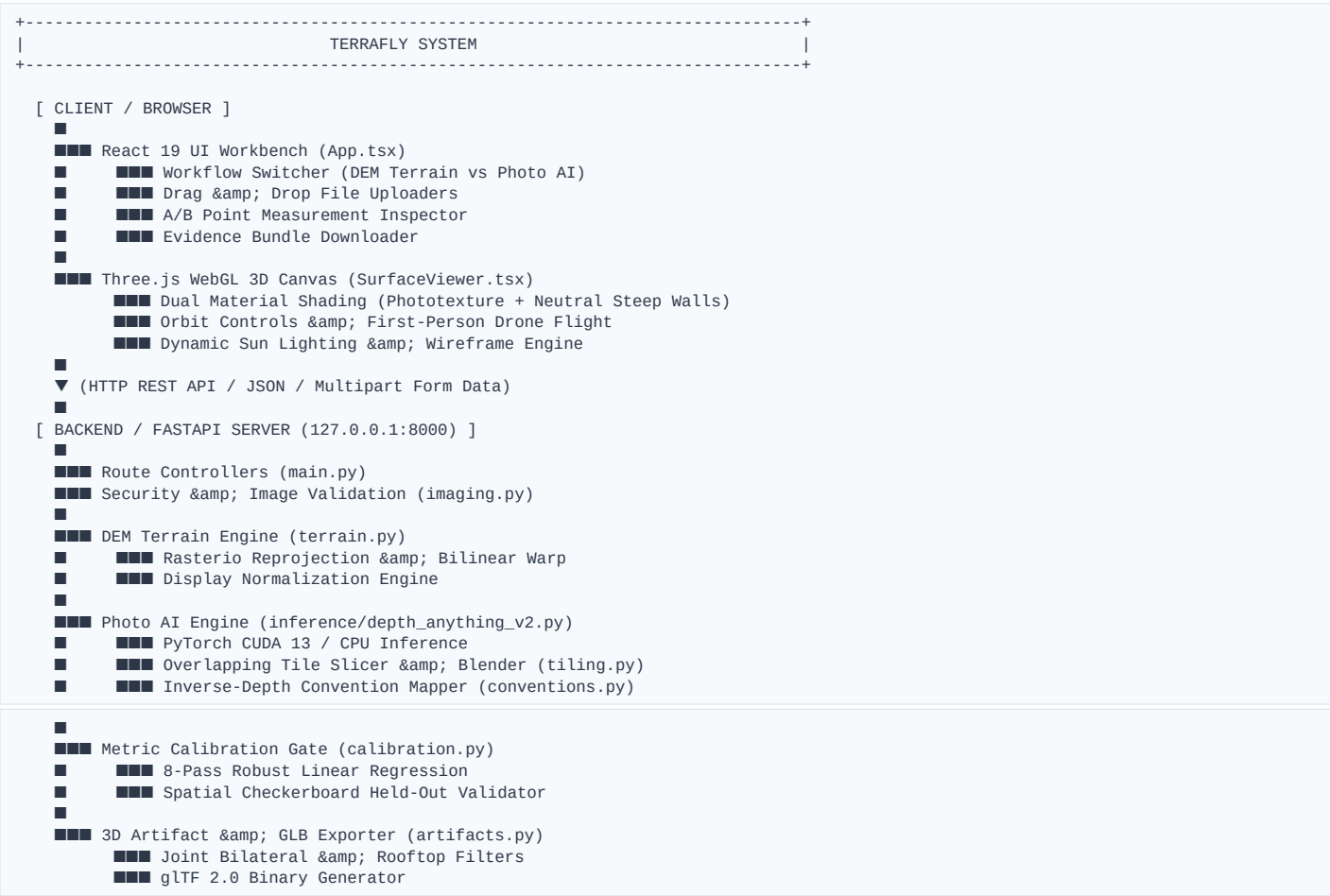
- The user clicks an adjacent mountain peak (sets **Point B**: Elevation $\$2,180.75\text{ m}\$$).
- The UI immediately displays:
- **Height difference**: $\$935.45\text{ m}\$$
- **Horizontal distance**: $\$1,850.20\text{ m}\$$
- **Terrain slope**: $\$26.8^\circ\circ$

8. Evidence Export:

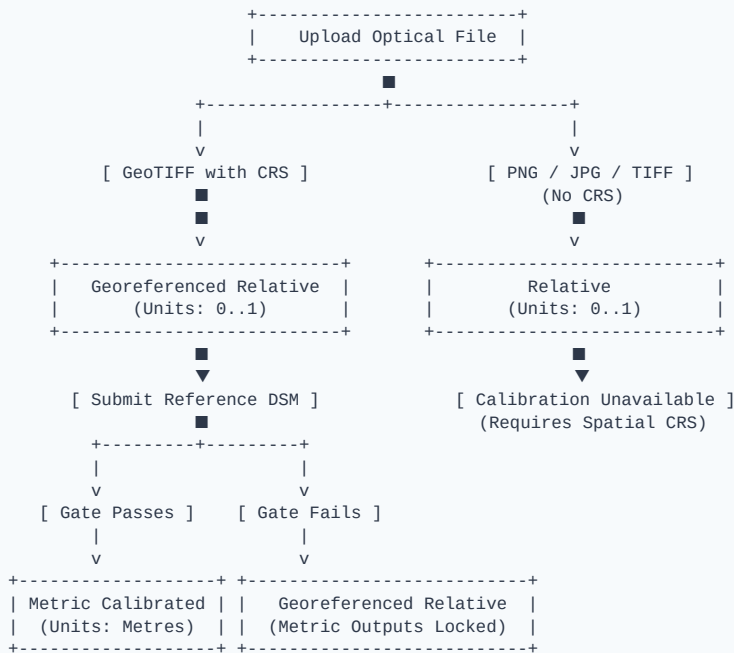
- The user expands the **Evidence bundle** and downloads the aligned **metric_surface.tif** and **terrain_alignment_report.json** containing complete SHA-256 verification hashes.

SECTION 15 — COMPLETE ARCHITECTURE DIAGRAMS

15.1 System Component Diagram



15.2 Scientific State Machine



SECTION 16 — HOW TO READ THIS CODEBASE YOURSELF

To master the TerraFly codebase without AI assistance, follow this recommended reading order:

- Start with Schemas** ([backend/terrafly/schemas.py](#)): Understand the data models ([JobManifest](#), [ScientificState](#), [Artifact](#), [Capabilities](#)). Everything in the backend revolves around these Pydantic models.
- Read the API Routes** ([backend/terrafly/main.py](#)): See how HTTP requests arrive, how files are validated, and how background tasks are dispatched.
- Inspect Image Sanitization** ([backend/terrafly/imaging.py](#)): Learn how raw bytes are safely checked for size, format, decompression bombs, and geospatial CRS.
- Follow the DEM Pipeline** ([backend/terrafly/terrain.py](#)): Understand how optical GeoTIFFs and DEM rasters are aligned, reprojected, and normalized.
- Follow the AI Pipeline** ([backend/terrafly/inference/depth_anything_v2.py](#) & [conventions.py](#)): Trace how PyTorch runs Depth Anything V2 and converts raw inverse-depth tensors into relative height arrays.
- Study the Calibration Math** ([backend/terrafly/calibration.py](#)): Trace the 8-pass robust linear regression, checkerboard split, and quality gate logic.
- Inspect Artifact Generation** ([backend/terrafly/artifacts.py](#)): See how binary [.glb](#) 3D meshes are constructed from NumPy height grids.
- Examine Frontend Geometry** ([frontend/src/surfaceGeometry.ts](#)): Learn how Three.js vertex positions, UV coordinates, and bilinear sampling are implemented.
- Examine the 3D Viewer** ([frontend/src/SurfaceViewer.tsx](#)): See how WebGL shaders, camera controls, lighting, and mouse raycasting function.
- Finish with the Workbench UI** ([frontend/src/App.tsx](#)): Understand how user state, polling loops, and UI views tie the entire system together.

SECTION 17 — HOW TO MODIFY TERRAFLY SAFELY

17.1 Modifying Safe vs. Critical Files

- **Safe to Modify (UI / Styling):**
 - **frontend/src/styles.css**: Modify colour tokens, fonts, spacing, and layout rules.
 - **frontend/src/App.tsx**: Add informational banners, reorganize toolbar buttons, adjust text labels.
- **Moderate Risk (Configuration & Limits):**
 - **backend/terrafly/config.py**: Adjust **max_upload_bytes**, **tile_size**, or default model IDs. Always test with **scripts/verify.ps1**.
- **Critical Files (DO NOT modify without running full test suite):**
 - **backend/terrafly/calibration.py**: Modifying regression or gating logic will break scientific correctness.
 - **backend/terrafly/inference/conventions.py**: Changing conversion logic will invert buildings into holes.
 - **backend/terrafly/artifacts.py**: Modifying GLB binary packing will corrupt 3D file exports.
 - **frontend/src/surfaceGeometry.ts**: Modifying sampling logic will break A/B point measurements.

17.2 Safe Environment Variable Overrides

Create a **.env** file in the root directory (based on **.env.example**):

```
TERRAFLY_DEVICE=cuda           # Force CUDA or CPU
TERRAFLY_MODEL_ID=depth-anything/Depth-Anything-V2-Large-hf
TERRAFLY_MAX_UPLOAD_BYTES=26214400 # 25 MB
TERRAFLY_MAX_WORKING_BYTES=805306368 # 768 MB
TERRAFLY_ALLOW_TEST_ADAPTER=0    # 1 only during automated tests
```

SECTION 18 — BUGS, TECHNICAL DEBT, & KNOWN LIMITATIONS

18.1 Summary of Fixed Issues in TerraFly 1.0

1. **Model Inversion Fix (Historical)**: The original implementation mapped Depth Anything V2 output using `$1 - \text{depth}$`, which accidentally inverted rooftops into craters. This was completely replaced with the explicit inverse-depth convention (`$z_{rel} = \text{normalized}$`), ensuring rooftops stay above ground.
2. **Separation of Canonical and Display Grids**: Early prototypes applied smoothing directly to the measurement array. TerraFly 1.0 cleanly separates **relative_surface.npy** (untouched) from **display_grid.json** (filtered for rendering).
3. **Texture Drip Fix**: Steep heightfield triangles now use a neutral synthetic wall material instead of stretching top-down aerial pixels vertically.
4. **One-Address Launcher Environment Fix**: Fixed an issue where inherited duplicate **PATH** variables on Windows prevented PowerShell from launching Uvicorn.

18.2 Current Limitations (Scientific Boundaries)

- **Monocular Scale Ambiguity**: A single 2D image without surveyed control points cannot establish true elevation in metres.
- **Perspective Tilt**: Overhead imagery taken at oblique angles contains global perspective tilt that mimics terrain slope. TerraFly detects and warns about this tilt but does not automatically flatten it to avoid altering real mountain slopes.
- **Single-Band Thermal Domain**: Grayscale or thermal infrared (TIR) imagery is accepted for compatibility, but users are warned that it falls outside the optical training distribution.

SECTION 19 — TERRAFLY IN ONE PAGE

+-----+ TERRAFLY IN ONE PAGE +-----+	
WHAT TERRAFLY DOES: Transforms 2D satellite/aerial images and DEMs into interactive, textured 3D terrain models with scientifically honest height, distance, and slope inspection.	
WHAT THE USER PROVIDES: • DEM Terrain Mode: Optical GeoTIFF + Single-band DEM GeoTIFF + Datum name. • Photo AI Mode: 2D aerial image (PNG/JPG/TIFF) + optional Reference DSM.	
WHAT HAPPENS INTERNALLY: 1. Validates file security, dimensions, memory budget, and geospatial CRS. 2. DEM Terrain: Reprojects DEM onto optical grid (bilinear warp, >=90% overlap). 3. Photo AI: Runs Depth Anything V2 ViT Large (CUDA/CPU with tiled inference). 4. Generates separate display mesh (neutral steep walls, spike cleanup). 5. Calibrates: 8-pass robust linear regression with held-out checkerboard gate.	
WHAT CALCULATIONS HAPPEN: • Bilinear sub-pixel sampling under 3D mouse clicks. • Height Delta: z_B - z_A (relative or metres). • Horizontal Distance: Euclidean distance in projected CRS metres. • Terrain Slope: arctan(Height Delta / Horizontal Distance) in degrees. • Held-Out Validation: RMSE, MAE, Bias, and R² against independent ground truth.	
WHAT AI DOES: Estimates relative monocular depth/structure (0.0 to 1.0) from a single 2D image.	
WHAT DETERMINISTIC MATH DOES: Everything else: DEM reprojection, calibration fitting, quality gates, 3D normals, A/B measurements, slope calculations, and GLB generation.	
WHAT RESULT IS PRODUCED: An interactive 3D WebGL viewer + downloadable evidence bundle (calibrated GeoTIFFs, lossless NumPy arrays, textured GLB models, and cryptographic audit manifests).	
+-----+	